Print

Relatório Final



Departamento de Engenharia Informática e de Sistemas

[nome do curso]

Base de Dados

2025/2026

Tema: Viagens Uber

Identificação dos Autores

Nº Aluno	Prática	Nome	Email
2024140853	P4	Gustavo Jesus	a2024140853@isec.pt
2024140881	P4	Miguel Rosário	a2024140881@isec.pt
2024140892	P4	Rodrigo Ideia	a2024140892@isec.pt
2024140909	P4	Tiago Carreira	a2024140909@isec.pt

Distribuição do esforço

Tarefa	Gustavo Jesus	Miguel Rosário	Rodrigo Ideia	Tiago Carreira
Documento específicação inicial	25%	25%	25%	25%
Desenvolvimento dos Modelos	25%	25%	25%	25%
Documento relatório final	25%	25%	25%	25%

Cap 1 - Introdução

O objetivo deste projeto é implementar um sistema que permita efetuar de forma automatizada o controlo de viagens da empresa Uber Technologies Inc..

A empresa Uber Technologies Inc. é uma empresa multinacional americana, prestadora de serviços eletrónicos na área do transporte privado urbano, através de uma aplicação de transporte que permite a busca por motoristas baseada na localização, oferecendo um serviço similar ao tradicional táxi.

Cap 2 - Enquadramento da Proposta

A Uber representa um sistema complexo que gere milhões de interações diárias entre motoristas e passageiros. O objetivo é desenhar uma estrutura que suporte as funcionalidades nucleares do negócio: o registo de utilizadores, a alocação de viagens, o processamento de pagamentos e as avaliações mútuas.

Diagnóstico da Situação Atual

Para o funcionamento de uma plataforma deste tipo, a ausência de uma base de dados traz problemas graves de gestão. Identificam-se falhas críticas como a Perda de Histórico, onde a incapacidade de armazenar o registo de viagens passadas impede a análise de faturação, e a falta de Integridade dos Dados, pois sem regras bem definidas, seria possível agendar uma viagem para um motorista que não existe.

Problemas encontrados

O primeiro problema identificado é a redundância e a inconsistência dos dados dos utilizadores. Sem uma base de dados relacional, as informações de um motorista ou passageiro teriam de ser repetidas em cada registo de serviço, o que levaria a erros e desatualizações frequentes. Esta dificuldade é resolvida através da criação de entidades únicas para Clientes e Motoristas. Desta forma, os dados pessoais são registados apenas uma vez e referenciados nas viagens através de um identificador único, garantindo que qualquer alteração (como mudar de telemóvel) se reflete automaticamente em todo o histórico.

Um segundo problema relevante é a desconexão entre a prestação do serviço e o registo financeiro. Na situação atual, é difícil validar se um pagamento corresponde exatamente à viagem realizada, o que pode gerar fraudes ou erros de contabilidade. Esta situação é resolvida através da associação obrigatória de pagamentos a reservas. A solução proposta obriga a que cada registo na tabela de pagamentos esteja vinculado a uma reserva existente, permitindo calcular saldos de forma automática e segura.

Por fim, um problema também detetado é a falta de controlo de qualidade e confiança entre desconhecidos. Sem um histórico acessível, não é possível distinguir bons de maus prestadores de serviço. Este problema é solucionado através da implementação de um sistema de avaliações persistente, onde as notas e comentários ficam gravados e associados a cada viagem, permitindo calcular a reputação média de motoristas e passageiros ao longo do tempo.

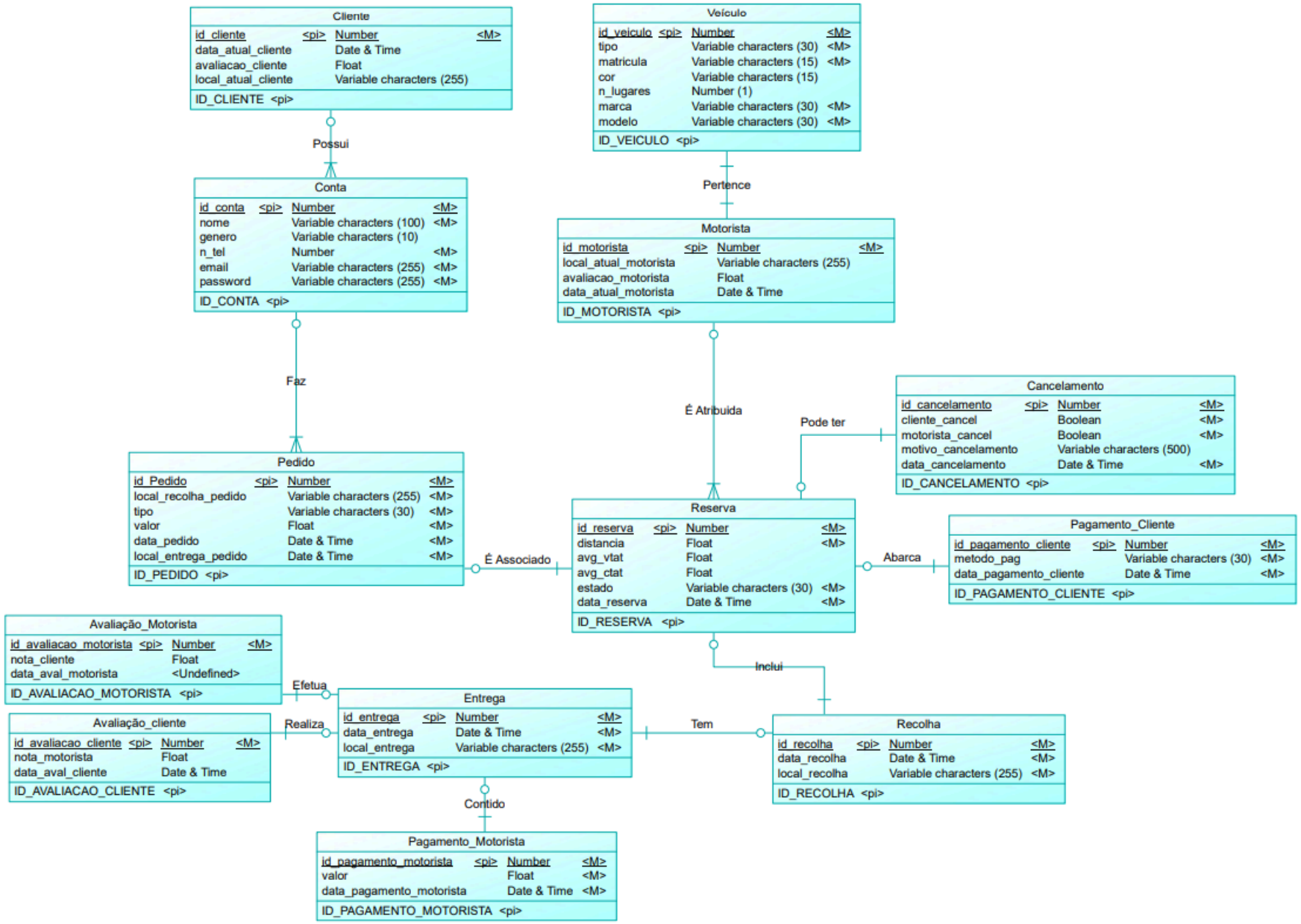
Descrição da Solução Proposta

A proposta deste trabalho é a modelação e implementação de uma Base de Dados Relacional Simplificada que resolva os problemas de organização de informação identificados acima.

Esta solução permitirá armazenar todos os dados, garantir que cada viagem está ligada a um passageiro, a um motorista e a um veículo registado, fornecer dados estruturados que permitem consultas rápidas sobre faturação, histórico de rotas e níveis de satisfação dos clientes.

Cap 3 - Modelo Conceptual

Diagrama do Modelo Conceptual


Cliente
id_cliente <pi> Number <M>
data_atual_cliente Date & Time
avaliacao_cliente Float
local_atual_cliente Variable characters (255)
ID_CLIENTE <pi>
Possui
Conta
id_conta <pi> Number <M>
nome Variable characters (100) <M>
genero Variable characters (10)
n_tel Number <M>
email Variable characters (255) <M>
password Variable characters (255) <M>
ID_CONTA <pi>
Faz
Pedido
id_Pedido <pi> Number <M>
local_recolha_pedido Variable characters (255) <M>
tipo Variable characters (30) <M>
valor Float <M>
data_pedido Date & Time <M>
local_entrega_pedido Date & Time <M>
ID_PEDIDO <pi>
É Associado
Veículo
id_veiculo <pi> Number <M>
tipo Variable characters (30) <M>
matricula Variable characters (15) <M>
cor Variable characters (15)
n_lugares Number (1)
marca Variable characters (30) <M>
modelo Variable characters (30) <M>
ID_VEICULO <pi>
Pertence
Motorista
id_motorista <pi> Number <M>
local_atual_motorista Variable characters (255)
avaliacao_motorista Float
data_atual_motorista Date & Time
ID_MOTORISTA <pi>
É Atribuida
Pode ter
Cancelamento
id_cancelamento <pi> Number <M>
cliente_cancel Boolean <M>
motorista_cancel Boolean <M>
motivo_cancelamento Variable characters (500)
data_cancelamento Date & Time <M>
ID_CANCELAMENTO <pi>
Reserva
id_reserva <pi> Number <M>
distancia Float <M>
avg_vtat Float
avg_ctat Float
estado Variable characters (30) <M>
data_reserva Date & Time <M>
ID_RESERVA <pi>
Abarca
Pagamento_Cliente
id_pagamento_cliente <pi> Number <M>
metodo_pag Variable characters (30) <M>
data_pagamento_cliente Date & Time <M>
ID_PAGAMENTO_CLIENTE <pi>
Inclui
Avaliação_Motorista
id_avaliacao_motorista <pi> Number <M>
nota_cliente Float
data_aval_motorista <Undefined>
ID_AVALIACAO_MOTORISTA <pi>
Efetua
Avaliação_cliente
id_avaliacao_cliente <pi> Number <M>
nota_motorista Float
data_aval_cliente Date & Time
ID_AVALIACAO_CLIENTE <pi>
Realiza
Entrega
id_entrega <pi> Number <M>
data_entrega Date & Time <M>
local_entrega Variable characters (255) <M>
ID_ENTREGA <pi>
Tem
Recolha
id_recolha <pi> Number <M>
data_recolha Date & Time <M>
local_recolha Variable characters (255) <M>
ID_RECOLHA <pi>
Contido
Pagamento_Motorista
id_pagamento_motorista <pi> Number <M>
valor Float <M>
data_pagamento_motorista Date & Time <M>
ID_PAGAMENTO_MOTORISTA <pi>

CONTA: A entidade que armazena os dados de acesso e contactos comuns a todos os clientes.

CLIENTE: Representa o utilizador que solicita o serviço, contendo a sua localização atual e avaliação.

MOTORISTA: Representa o prestador de serviço.

VEÍCULO: Contém todos os dados do veículo (matrícula, marca, modelo)

PEDIDO: Regista a intenção inicial de viagem por parte do cliente, indicando o local de recolha.

RESERVA: A entidade central que concretiza o serviço, associando um Motorista a um Pedido.

RECOLHA E ENTREGA: Entidades operacionais que registam os momentos e locais exatos do início e fim da viagem.

PAGAMENTO: Entidades distintas (Pagamento Cliente e Pagamento Motorista) para registar os fluxos financeiros de entrada e saída.

AVALIAÇÃO: Entidades responsáveis por armazenar o feedback de ambas as partes.

CANCELAMENTO: Entidades responsáveis por armazenar o feedback de ambas as partes em relação ao cancelamento.

Relacionamento:

Cliente e Conta (1:N): Um cliente pode ter várias contas, mas uma conta tem de ser obrigatoriamente de um cliente.

Motorista e Veículo (1:1): Um motorista tem um veículo a uso, ambos obrigatórios.

Conta e Pedido (1:N): Uma conta pode efetuar vários pedidos de viagem, no entanto um pedido é obrigatório ser de uma conta.

Pedido e Reserva (1:1): Um pedido aceite gera uma única reserva, mas para haver uma reserva tem de haver obrigatóriamente um pedido.

Motorista e Reserva (1:N): Um motorista pode ser associado a várias reserva mas uma reserva tem de ter obrigatóriamente um motorista.

Reserva e Cancelamento (1:1): Uma reserva poderá ser cancelada, e um cancelamento tem de ter obrigatóriamente uma reserva.

Reserva e Recolha (1:1): Uma reserva gera uma recolha e uma recolha está obrigatóriamente associada a uma reserva.

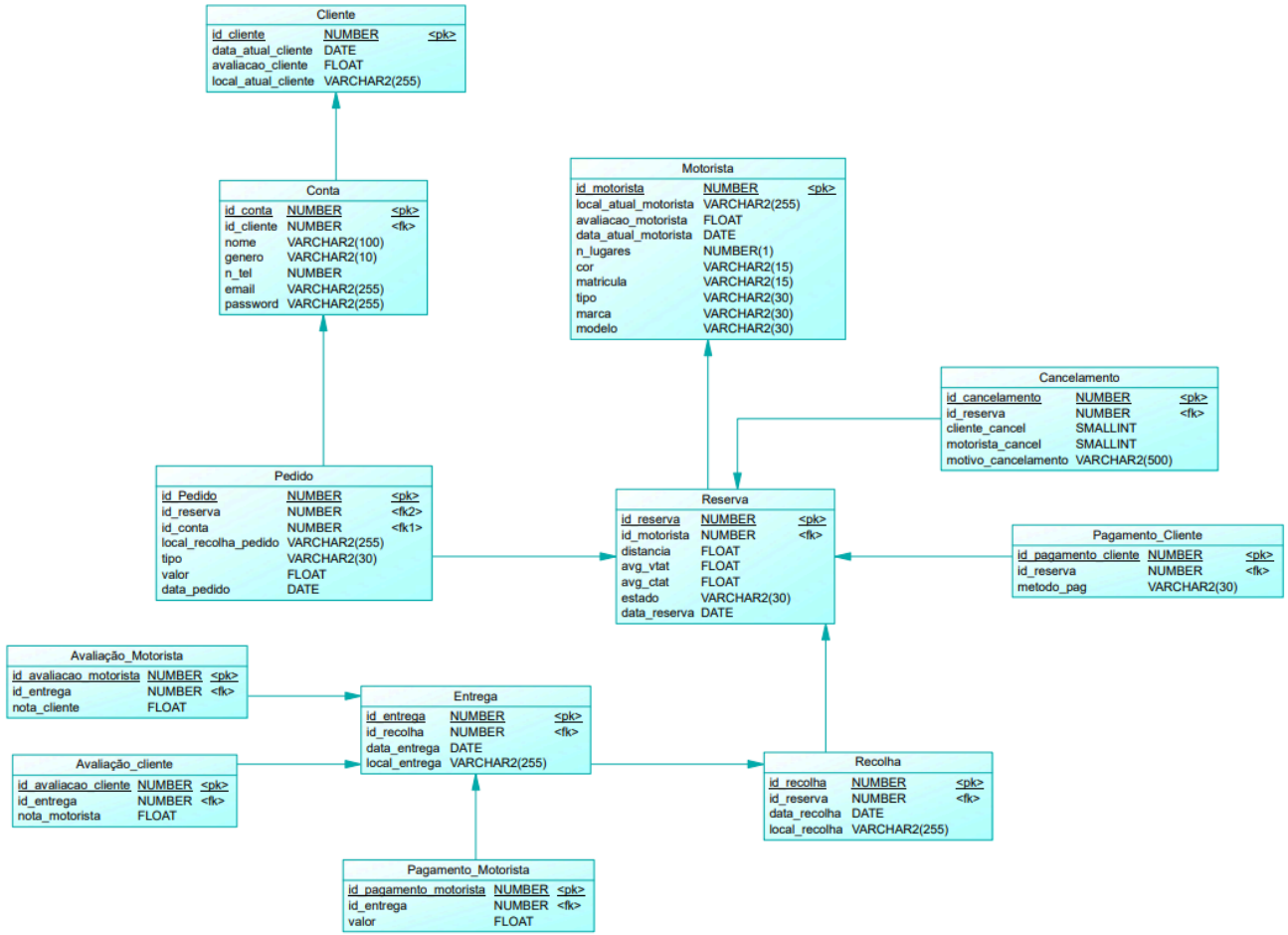
Recolha e Entrega (1:1): Uma recolha gera uma entrega e uma entrega está obrigatóriamente associada a uma recolha.

Entrega e Pagamento Motorista (1:1): Cada serviço concluído gera um registo financeiro único, sendo este o pagamento autorizado para o motorista, e este pagamento necessita obrigatóriamente de uma entrega.

Entrega e Avaliação (Motorista e Cliente) (1:1): Após a conclusão da entrega, podem ser gerados registos de avaliação (do motorista e do cliente), estas avaliações necessitam obrigatóriamente de uma entrega.

Cap 4 - Modelo Físico

Diagrama do Modelo Físico



Script de criação da Base de Dados

```
/*==============================================================*/
/* DBMS name:      ORACLE Version 11g                           */
/* Created on:     09/12/2025 12:16:54                          */
/*==============================================================*/


alter table AVALIACAO_CLIENTE
   drop constraint FK_AVALIACA_REALIZA_ENTREGA;

alter table AVALIACAO_MOTORISTA
   drop constraint FK_AVALIACA_EFETUA_ENTREGA;

alter table CANCELAMENTO
   drop constraint FK_CANCELAM_PODE_TER2_RESERVA;

alter table CONTA
   drop constraint FK_CONTA_POSSUI_CLIENTE;

alter table ENTREGA
   drop constraint FK_ENTREGA_TEM_RECOLHA;

alter table PAGAMENTO_CLIENTE
   drop constraint FK_PAGAMENT_ABARCA_RESERVA;

alter table PAGAMENTO_MOTORISTA
   drop constraint FK_PAGAMENT_CONTIDO_ENTREGA;

alter table PEDIDO
   drop constraint FK_PEDIDO_E_ASSOCIA_RESERVA;

alter table PEDIDO
   drop constraint FK_PEDIDO_FAZ_CONTA;

alter table RECOLHA
   drop constraint FK_RECOLHA_INCLUI2_RESERVA;
```

```
alter table RESERVA
   drop constraint FK_RESERVA_E_ATRIBUI_MOTORIST;

drop table AVALIACAO_CLIENTE cascade constraints;

drop table AVALIACAO_MOTORISTA cascade constraints;

drop table CANCELAMENTO cascade constraints;

drop table CLIENTE cascade constraints;

drop table CONTA cascade constraints;

drop table ENTREGA cascade constraints;

drop table MOTORISTA cascade constraints;

drop table PAGAMENTO_CLIENTE cascade constraints;

drop table PAGAMENTO_MOTORISTA cascade constraints;

drop table PEDIDO cascade constraints;

drop table RECOLHA cascade constraints;

drop table RESERVA cascade constraints;

/*==============================================================*/
/* Table: AVALIACAO_CLIENTE                                     */
/*==============================================================*/
create table AVALIACAO_CLIENTE
(
   ID_AVALIACAO_CLIENTE NUMBER              not null,
   ID_ENTREGA           NUMBER,
   NOTA_MOTORISTA       FLOAT,
   constraint PK_AVALIACAO_CLIENTE primary key (ID_AVALIACAO_CLIENTE)
```

```
);

/*==============================================================*/
/* Table: AVALIACAO_MOTORISTA                                   */
/*==============================================================*/
create table AVALIACAO_MOTORISTA
(
  ID_AVALIACAO_MOTORISTA NUMBER               not null,
  ID_ENTREGA          NUMBER,
  NOTA_CLIENTE        FLOAT,
  constraint PK_AVALIACAO_MOTORISTA primary key (ID_AVALIACAO_MOTORISTA)
);

/*==============================================================*/
/* Table: CANCELAMENTO                                          */
/*==============================================================*/
create table CANCELAMENTO
(
  ID_CANCELAMENTO     NUMBER              not null,
  ID_RESERVA          NUMBER,
  CLIENTE_CANCEL      SMALLINT            not null,
  MOTORISTA_CANCEL    SMALLINT            not null,
  MOTIVO_CANCELAMENTO  VARCHAR2(500),
  constraint PK_CANCELAMENTO primary key (ID_CANCELAMENTO)
);

/*==============================================================*/
/* Table: CLIENTE                                               */
/*==============================================================*/
create table CLIENTE
(
  ID_CLIENTE          NUMBER              not null,
  DATA_ATUAL_CLIENTE   DATE,
  AVALIACAO_CLIENTE    FLOAT,
  LOCAL_ATUAL_CLIENTE  VARCHAR2(255),
  constraint PK_CLIENTE primary key (ID_CLIENTE)
```

```
);

/*==============================================================*/
/* Table: CONTA                                                 */
/*==============================================================*/
create table CONTA
(
   ID_CONTA            NUMBER              not null,
   ID_CLIENTE          NUMBER,
   NOME                VARCHAR2(100)       not null,
   GENERO              VARCHAR2(10),
   N_TEL               NUMBER              not null,
   EMAIL               VARCHAR2(255)       not null,
   PASSWORD            VARCHAR2(255)       not null,
   constraint PK_CONTA primary key (ID_CONTA)
);

/*==============================================================*/
/* Table: ENTREGA                                               */
/*==============================================================*/
create table ENTREGA
(
   ID_ENTREGA          NUMBER              not null,
   ID_RECOLHA          NUMBER,
   DATA_ENTREGA        DATE                not null,
   LOCAL_ENTREGA       VARCHAR2(255)       not null,
   constraint PK_ENTREGA primary key (ID_ENTREGA)
);

/*==============================================================*/
/* Table: MOTORISTA                                             */
/*==============================================================*/
create table MOTORISTA
(
   ID_MOTORISTA        NUMBER              not null,
   LOCAL_ATUAL_MOTORISTA VARCHAR2(255),
```

```
  AVALIACAO_MOTORISTA  FLOAT,
  DATA_ATUAL_MOTORISTA DATE,
  N_LUGARES           NUMBER(1),
  COR                 VARCHAR2(15),
  MATRICULA           VARCHAR2(15),
  TIPO                VARCHAR2(30),
  MARCA               VARCHAR2(30),
  MODELO              VARCHAR2(30),
  constraint PK_MOTORISTA primary key (ID_MOTORISTA)
);

/*==============================================================*/
/* Table: PAGAMENTO_CLIENTE                                     */
/*==============================================================*/
create table PAGAMENTO_CLIENTE
(
  ID_PAGAMENTO_CLIENTE NUMBER              not null,
  ID_RESERVA          NUMBER,
  METODO_PAG          VARCHAR2(30)        not null,
  constraint PK_PAGAMENTO_CLIENTE primary key (ID_PAGAMENTO_CLIENTE)
);

/*==============================================================*/
/* Table: PAGAMENTO_MOTORISTA                                   */
/*==============================================================*/
create table PAGAMENTO_MOTORISTA
(
  ID_PAGAMENTO_MOTORISTA NUMBER              not null,
  ID_ENTREGA          NUMBER,
  VALOR               FLOAT               not null,
  constraint PK_PAGAMENTO_MOTORISTA primary key (ID_PAGAMENTO_MOTORISTA)
);

/*==============================================================*/
/* Table: PEDIDO                                                */
/*==============================================================*/
```

```
create table PEDIDO
(
   ID_PEDIDO            NUMBER               not null,
   ID_RESERVA           NUMBER               not null,
   ID_CONTA             NUMBER,
   LOCAL_RECOLHA_PEDIDO VARCHAR2(255)        not null,
   TIPO                 VARCHAR2(30)         not null,
   VALOR                FLOAT                not null,
   DATA_PEDIDO          DATE                 not null,
   constraint PK_PEDIDO primary key (ID_PEDIDO)
);

/*==============================================================*/
/* Table: RECOLHA                                               */
/*==============================================================*/
create table RECOLHA
(
   ID_RECOLHA           NUMBER               not null,
   ID_RESERVA           NUMBER               not null,
   DATA_RECOLHA         DATE                 not null,
   LOCAL_RECOLHA        VARCHAR2(255)        not null,
   constraint PK_RECOLHA primary key (ID_RECOLHA)
);

/*==============================================================*/
/* Table: RESERVA                                               */
/*==============================================================*/
create table RESERVA
(
   ID_RESERVA           NUMBER               not null,
   ID_MOTORISTA         NUMBER,
   DISTANCIA            FLOAT                not null,
   AVG_VTAT             FLOAT,
   AVG_CTAT             FLOAT,
   ESTADO               VARCHAR2(30)         not null,
   DATA_RESERVA         DATE                 not null,
```

```
  constraint PK_RESERVA primary key (ID_RESERVA)
);

alter table AVALIACAO_CLIENTE
  add constraint FK_AVALIACA_REALIZA_ENTREGA foreign key (ID_ENTREGA)
    references ENTREGA (ID_ENTREGA);

alter table AVALIACAO_MOTORISTA
  add constraint FK_AVALIACA_EFETUA_ENTREGA foreign key (ID_ENTREGA)
    references ENTREGA (ID_ENTREGA);

alter table CANCELAMENTO
  add constraint FK_CANCELAM_PODE_TER2_RESERVA foreign key (ID_RESERVA)
    references RESERVA (ID_RESERVA);

alter table CONTA
  add constraint FK_CONTA_POSSUI_CLIENTE foreign key (ID_CLIENTE)
    references CLIENTE (ID_CLIENTE);

alter table ENTREGA
  add constraint FK_ENTREGA_TEM_RECOLHA foreign key (ID_RECOLHA)
    references RECOLHA (ID_RECOLHA);

alter table PAGAMENTO_CLIENTE
  add constraint FK_PAGAMENT_ABARCA_RESERVA foreign key (ID_RESERVA)
    references RESERVA (ID_RESERVA);

alter table PAGAMENTO_MOTORISTA
  add constraint FK_PAGAMENT_CONTIDO_ENTREGA foreign key (ID_ENTREGA)
    references ENTREGA (ID_ENTREGA);

alter table PEDIDO
  add constraint FK_PEDIDO_E_ASSOCIA_RESERVA foreign key (ID_RESERVA)
    references RESERVA (ID_RESERVA);

alter table PEDIDO
```

```
  add constraint FK_PEDIDO_FAZ_CONTA foreign key (ID_CONTA)
    references CONTA (ID_CONTA);

alter table RECOLHA
  add constraint FK_RECOLHA_INCLUI2_RESERVA foreign key (ID_RESERVA)
    references RESERVA (ID_RESERVA);

alter table RESERVA
  add constraint FK_RESERVA_E_ATRIBUI_MOTORIST foreign key (ID_MOTORISTA)
    references MOTORISTA (ID_MOTORISTA);
```

Cap 5 - Conclusões

A implementação da base de dados representa uma mudança face aos processos de gestão manuais e ineficientes.

As principais contribuições desta proposta para o funcionamento do negócio são:

1. **Integridade e Centralização da Informação:** Garante-se que a informação está sempre atualizada e acessível, resolvendo os problemas de inconsistência que afetavam a gestão de utilizadores.
2. **Rastreabilidade Operacional e Financeira:** Permite à empresa calcular com precisão as receitas e validar pagamentos, algo impossível de realizar eficientemente sem este suporte tecnológico.
3. **Suporte à Tomada de Decisão:** O armazenamento histórico de todos os pedidos e avaliações transforma dados brutos em conhecimento. A solução permite identificar os motoristas com melhor desempenho e os clientes mais frequentes, dotando a gestão de ferramentas para premiar a qualidade e melhorar o serviço.

Este projeto permitiu adquirir conhecimentos sólidos sobre como as bases de dados funcionam na prática. Foi possível aprender a armazenar dados de forma eficiente, escolhendo os tipos de dados corretos e definindo chaves primárias e estrangeiras que impedem a corrupção da informação, destacou também a importância de saber gerir uma empresa na perspetiva da base de dados. Percebeu-se que as regras de negócio (como pagamentos e atribuição de serviços) têm de ser traduzidas em regras de dados (restrições e relacionamentos), provando que uma boa gestão empresarial depende diretamente de uma arquitetura de dados bem desenhada.

Referências Bibliográficas

Anexos